

Boot Procedure for an ESP32 SOC

During the development of our own custom [ESP32 development board](#) we learned a few things through a combination of reading manuals as well as making observations. This article captures this information in case it proves helpful in the future.

ESPTOOL.PY

Esptool.py is a Python-based, open source, platform independent serial communication tool provided by Espressif and used by IDEs like PlatformIO to communicate with the ESP32 ROM boot loader. By default it operates at a baud rate of 115200. Espressif hosts pages that act as a [manual for this tool](#).

ESP32 Strapping Pins

There are a total of five strapping pins that control how the ESP32 behaves at startup. Of those five pins the only two that matter for our purposes are GPIO0 (physical pin 25) and GPIO2 (physical pin 24). At boot time GPIO0 (physical pin 25) must be pulled low. GPIO2 (physical pin 24) is ignored in normal boot mode but to play it safe pull this pin LOW as well.

The other strapping pins control operating voltage for the chip as well as putting the chip in silent boot mode. We just leave these pins in their default setting to get 3.3V and boot messages echoed out.

The ESP32 has a 45k ohm internal pull-up/pull-down resistor at GPIO0 (and other pins). If you want to connect a switch button to enter the boot mode, this has to be a strong pull-down. For example a 10k resistor to GND.

Boot Log

If you have a serial console active during the booting of an ESP32, and the MTDO strapping pin is not driven low (it is high by default via an internal 45K ohm resistor) then you will see the boot log for the ESP32.

TO DO:

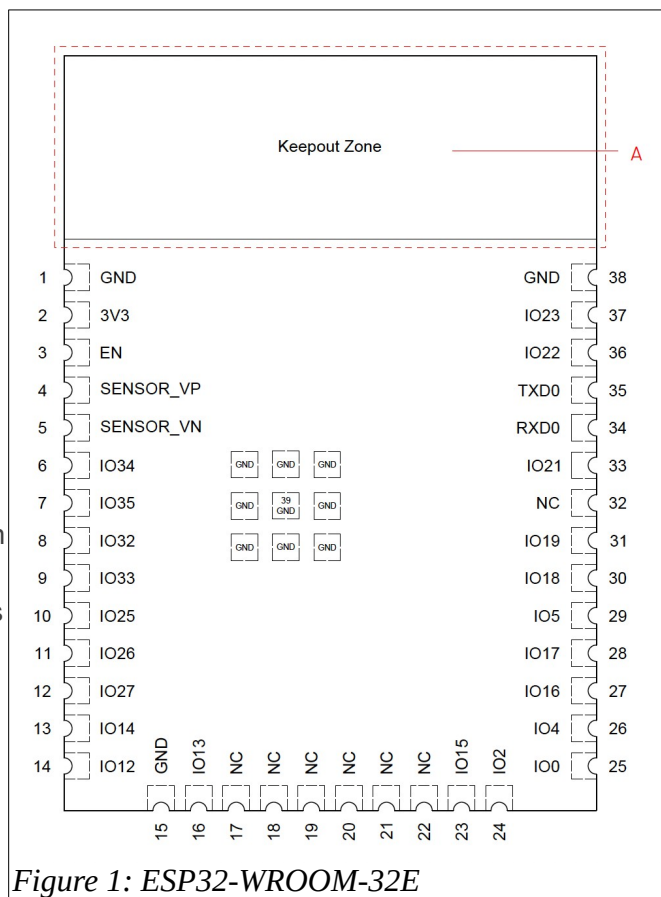


Figure 1: ESP32-WROOM-32E

We need to update this section with what we are seeing. For example, we can see messages like “boot load, waiting” messages.

For some reason even without the RTS and DTS circuitry we saw our board automatically load firmware and reboot and run the code without ever touching the reset button.. We need to figure out what is going on there.

Use [this link](#) to help understand what is happening and update this doc..

Boot sequence

This section explains the steps which happen before the `app_main` (our firmware binary) function of an ESP-IDF application is called.

The high level view of startup process is as follows:

1. **First stage (ROM) bootloader** loads the second stage bootloader image to RAM (IRAM & DRAM) from flash offset 0x1000.
2. **Second Stage Bootloader** loads partition table and main app image from flash. Main app incorporates both RAM segments and read-only segments mapped via flash cache.
3. **Application Startup** executes. At this point, the second CPU and RTOS scheduler are started, which then run the `main_task`, leading to the execution of `app_main`.

Boot Timing

There are timing parameters that can be altered which control when certain events occur during the boot up sequence of the ESP32 SOC. By default the chips power on reset parameters are set as follows:

1. `tSTBL` determines the time that is allowed for the power rails to stabilize to their target voltage. This is set to 50uS by default.
2. `tRST` determines the time allowed to read the values of the strapping pins. This is set to 50uS by default.

Manual Booting

The minimal circuitry required to be able to enter boot mode is to have your board have one normally open switch connect to GPIO0 (physical pin 25) and one normally open switch connected to the EN button ([physical pin 3).

To use this configuration to load new firmware into the ESP32, hold down the GPIO0 button, press and release the EN button, wait 1 second, release the GPIO0 button.

Automatic booting

PlatformIO uses a python tool called esptool.py that is capable of automatically resetting our development board into bootloader mode. To achieve this we have to build a circuit that manages certain ESP32 “strapping pins” in order to get the correct boot mode.

The esptool.py program resets the ESP32 SOC automatically by asserting the DTR and RTS control lines of the USB your serial converter chip. On our board we use an FTDI TTL serial adapter chip. The DTR and RTS control lines are in turn connected to the GPIO0 and EN pins of ESP32. This allows the esptool.py utility to change the voltage levels of the DTR and RTS which in turn puts the ESP32 into Firmware Download mode.

Using a TTL Serial Adapter

While developing a new ESP32 development board you may wish to use an off the shelf [TTL Serial Adapter](#) to bypass your circuit to test if the ESP32 SOC is in good working order. To do this you can:

1. Make sure that you set the output voltage of the TTL serial adapter to 3.3V.
2. Connect the VCC pin from the TTL adapter to the 3.3V pin of the ESP32 (physical pin 3).
3. Connect the ground pin from the TTL adapter to one of the GND pins of the ESP32 (physical pin 1 for example).
4. Connect the EN pin to a normally open switch that also connects to ground.
5. Connect the GPIO0 (physical pin 25) to ground.
6. Connect the GPIO2 (physical pin 24) to ground. This is an optional step but eliminates any chance that your circuit is pulling it high.

Once the loading is complete you stop pulling GPIO0 low and press EN again to get the new firmware to run.